Microsoft Purview Automation Framework

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Automation Framework 5

The problem . . . 5

The solution . . . 5

What you get . . . 5

Approach . . . 5

Boundaries . . . 5

The bottom line . . . 6

1. Solution Overview — Microsoft Purview Automation Framework 6

What this solution is . . . 6

Why this matters . . . 6

How it works . . . 7

The Purview objects this framework manages . . . 7

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Microsoft Purview Automation Framework 9

2.1 Compliance licensing for the policy types you automate . . . 9

2.2 Purview roles (least-privilege) for each policy type . . . 9

2.3 Security & Compliance PowerShell + Graph toolchain . . . 10

2.4 A reviewed desired-state config (`Automation Config – Purview Ops – Baseline`) . . . 10

2.5 In-scope locations/workloads and a classification intent . . . 10

2.6 Compliance, records, privacy & legal ownership of policy content . . . 10

2.7 A pilot scope, change control, and understanding of the safe-first modes . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 13

3. Implementation Guide — Microsoft Purview Automation Framework 13

Step 1 — Confirm the toolchain, roles, and licensing (Assess) . . . 13

Step 2 — Inventory the current tenant state (Assess) . . . 14

Step 3 — Author and validate the desired-state config (Design) . . . 14

Step 4 — Deploy the pilot into safe-first mode (Configure → Pilot) . . . 14
Step 5 — Validate deployed state against desired state (Pilot → Validate) . . . 15
Step 6 — Report the pilot posture (Validate) . . . 15
Step 7 — Enforce/publish the pilot deliberately (Expand — gated) . . . 15
Step 8 — Expand to full scope (Expand) . . . 15
Step 9 — Operationalize the cadence (Expand) . . . 16
5. Testing & Validation — Microsoft Purview Automation Framework 16
Gate 1 — Toolchain, roles, and licensing are ready . . . 16
Gate 2 — Current-state snapshot is produced . . . 16
Gate 3 — Config validates structurally . . . 16
Gate 4 — Dry-run (`-WhatIf`) shows the intended actions only . . . 17
Gate 5 — Safe-first deploy lands objects in the intended mode . . . 17
Gate 6 — Validation reports no unexpected drift . . . 17
Gate 7 — DLP test/simulation results are reviewed . . . 17
Gate 8 — Reports/scorecard reflect reality . . . 17
Gate 9 — Enforcement is deliberate and reversible (only when enforcing) . . . 18
Gate 10 — Cadence catches injected drift . . . 18
Evidence to retain . . . 18
6. Troubleshooting — Microsoft Purview Automation Framework 18
`OPSAUTO-Health.ps1` shows a red item . . . 19
`-ValidateConfigOnly` fails or flags entries . . . 19
`-WhatIf` prints changes I didn't expect . . . 19
A policy deployed but in the wrong mode / already enforcing . . . 19
`OPSAUTO-Validate.ps1` reports drift . . . 19
`OPSAUTO-Report.ps1` scorecard shows present < expected, or unexpected enforced count . . . 20
`-Enforce -Commit` caused user/data impact you didn't want . . . 20
`-Remove -Commit` didn't delete a policy I expected . . . 20
Scheduled read-only chain didn't run / drift caught late . . . 20
`Connect-IPPSSession` / `Connect-MgGraph` fails mid-run . . . 21
When to escalate beyond this solution . . . 21
7. Best Practices — Automating Microsoft Purview Operations Safely 21
Treat policy as reviewable code . . . 21
Deploy safe-first, enforce deliberately . . . 21
Pilot small, expand under control . . . 21
Make drift visible . . . 22
Manage only what you own — and know the boundary . . . 22

Least-privilege and separation 22
Keep decisions where they belong 22
No secrets, ever 22
8. Deliverables Summary — Microsoft Purview Automation Framework 23
Documentation 23
PowerShell tools (5) 23
Named artifacts created / produced 24
Definition of done 24
Appendix A — PowerShell Scripts 26

Executive Summary — Microsoft Purview Automation Framework

The problem

Microsoft Purview administration done by hand does not scale and does not repeat. Different admins create policies differently, DLP and retention rules drift from what was designed, a label that should exist is quietly missing, and when an auditor asks to see every policy and its current state the answer is a manual scavenger hunt across an evolving portal. Manual Purview operations are a **compliance gap and a data-risk exposure** — hard to reproduce, hard to review, hard to prove.

The solution

A **configuration-as-code automation framework** for Purview operations. A single **declarative desired-state config** describes the DLP policies, sensitivity labels, and retention policies the organization intends to run; **PowerShell tooling** then **discovers** current tenant state, **deploys** the config **into simulation/test/unpublished mode first**, **validates** deployed state against the config (drift detection), and **reports** a policy inventory and coverage scorecard as evidence. Five safe-by-default PowerShell tools (read-only or `-WhatIf` until you commit; nothing enforces without a separate, deliberate `-Enforce -Commit`) run the health check, discovery, deployment, validation, and reporting. It uses only the Purview portal, Security & Compliance PowerShell, and Microsoft Graph — no third-party tooling, no secrets.

What you get

- **Policy as reviewable code** — DLP, label, and retention policies defined in a config file that is versionable, reviewable in change control, identical across pilot and production, and rebuildable after a mistake.
- **Safe-by-default deployment** — everything lands in DLP **test/simulation mode**, labels **unpublished**, retention **pilot-scoped** on deploy; **enforcement is a separate, explicit, committed action**.
- **Drift caught, not discovered late** — scheduled desired-state validation turns a silently edited or disabled policy into a reported difference.
- **Evidence on demand** — a policy inventory and coverage scorecard produced by script, not assembled by hand before every audit.
- **A repeatable operating rhythm** — a scheduled read-only chain (discover → validate → report) so posture stays known between changes.

Approach

Delivery follows a disciplined, reversible model — **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce)** — deploying **pilot-first** into test/unpublished mode on a bounded scope, with pass/fail gates and documented rollback before anything enforces tenant-wide.

Boundaries

The framework **carries** compliance decisions faithfully and **proves** they landed; it does **not make** them. *Which* sensitive info types, *which* retention duration, *which* labels, and whether a policy is legally adequate are **deferred to compliance, records, privacy, and legal**. Microsoft Purview is **licensing-/plan-dependent** (advanced DLP, auto-labeling, and several retention features require E5 / E5 Compliance; some are preview) and **actively evolving** — capabilities and cmdlet parameters, notably DLP simulation/mode values, are cited or flagged **"verify your licensing/tenant,"** never invented. Records- management, insider-risk, and communication-compliance objects are **out of automated scope**.

The bottom line

Purview operations that are **repeatable, reversible, and evidenced** — policy defined as code, deployed in simulation first, validated for drift, and reported on demand — so the compliance posture is **defensible, not a manual scramble.**

1. Solution Overview — Microsoft Purview Automation Framework

What this solution is

Purview administration done by hand does not scale and does not repeat. Policies get created differently by different admins, DLP rules drift from what was designed, a label that should exist in test is quietly missing, and when an auditor asks "show me every DLP/retention/label policy and its current state" the answer is a manual scavenger hunt across the portal. This solution delivers a **configuration-as-code automation framework** for Microsoft Purview operations: a **declarative desired-state config** describes the DLP policies, sensitivity labels, and retention policies your organization intends to run; **PowerShell tooling** then **discovers** the current tenant state, **deploys** the config **into simulation/test/ unpublished mode first**, **validates** deployed state against the desired config (drift detection), and **reports** a coverage scorecard and policy inventory as evidence. It uses only the **Microsoft Purview portal**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** — no third-party tooling and no secrets in code.

This is a **framework**, not a specific policy pack: it operationalizes *how* Purview policy objects are deployed, reviewed, and evidenced repeatably. *What* each policy must contain (which sensitive info types, which retention schedule, which labels) remains a compliance/records/legal determination — the framework carries those decisions faithfully into the tenant and proves they landed; it does not make them.

Why this matters

- **Repeatability is the whole point.** A DLP or retention policy created by clicking is hard to reproduce, review, or roll back. A policy defined in a config file and deployed by script is reviewable in change control, identical across pilot and production, and rebuildable after a mistake.
- **Purview changes are high-impact and must be piloted.** DLP, sensitivity-label, and retention changes affect real users and real data the moment they enforce. Every Purview policy type supports a **safer-than-enforce** first state — DLP **test/simulation mode**, labels **created-but-unpublished**, retention **created-but-scoped/unpublished** — and this framework deploys into that state **by default**, only enforcing on an explicit, separate, committed action.

• **Drift is invisible until it bites.** Someone edits a rule in the portal, a policy gets disabled "temporarily," a label falls out of a label policy — and nobody notices until an incident or audit. Scheduled **desired-state validation** turns silent drift into a caught, reported difference.

• **Evidence on demand.** A **policy inventory** and **coverage scorecard** produced by script — rather than assembled by hand before every audit — is what makes the compliance posture defensible.

How it works

1. **Establish the desired-state config.** A single, human-readable JSON config — `Automation Config – Purview Ops – Baseline` — lists the DLP policies, sensitivity labels, and retention policies that should exist, each with the attributes the framework manages (name, scope/locations, mode, and the identifying settings). Compliance/records/legal own the *content*; the config is the reviewable record of it.

2. **Discover the current state.** `OPSAUTO-Discovery.ps1` (read-only) inventories what actually exists in the tenant — DLP policies and their **mode**, sensitivity labels and label policies, retention policies and rules — and exports a state snapshot.

3. **Deploy into a safe first state.** `OPSAUTO-Deploy.ps1` reconciles config → tenant. It runs `-WhatIf` **by default** (prints the exact `New-/Set-*` it would run), creates DLP policies in **test/simulation mode**, labels **unpublished**, and retention **unenforced/pilot-scoped** on `-Commit`, and only moves an object to **enforce** on a deliberate, separate `-Enforce -Commit`.

4. **Validate against desired state.** `OPSAUTO-Validate.ps1` (read-only) compares deployed objects to the config and reports **drift** — missing, extra, mode-mismatched, or altered policies — with a pass/fail gate.

5. **Report the posture.** `OPSAUTO-Report.ps1` (read-only) produces the **policy inventory** and a **coverage scorecard** (what exists, in what mode, against what the config expects) as the audit-ready evidence artifact.

6. **Run on a cadence.** The read-only chain (Discovery → Validate → Report) is scheduled so drift and gaps surface early; Deploy is run under change control when the config changes.

The Purview objects this framework manages

• **DLP compliance policies & rules** — `*-DlpCompliancePolicy` / `*-DlpComplianceRule`, including the policy **mode** (test/simulation vs enforce). **Verify your tenant** for the exact mode values it accepts.

• **Sensitivity labels & label policies** — `*-Label` / `*-LabelPolicy` (labels created before they are published; publishing is the enforce step).

• **Retention (data lifecycle) policies & rules** — `*-RetentionCompliancePolicy` / `*-RetentionComplianceRule` (created and pilot-scoped before tenant-wide publish).

• All are **Security & Compliance PowerShell** objects (`Connect-IPPSSession`); **Graph** is used for licensing and reporting reads. **Records-management file plans, insider-risk, and communication-compliance objects are intentionally out of scope for automated deploy** — they carry legal/HR/records determinations and are managed under their own governance (see Limitations).

What you can control

• The **desired-state config** — which DLP/label/retention policies the framework knows about and their managed attributes.

• The **deploy mode** — nothing enforces without an explicit, separate `-Enforce -Commit`; the default deploy lands everything in test/simulation/unpublished.

• The **pilot scope** — `Automation Pilot – Ops Scope – Stage1` (a bounded set of locations/users/policies) validated before any enforce/publish.

• The **validation and reporting cadence** — how often drift is checked and the scorecard regenerated.

Limitations (be honest with buyers)

• **The framework carries decisions; it does not make them.** *Which* sensitive info types, *which* retention duration, *which* labels — and whether a policy is legally adequate — are **compliance/records/privacy/legal determinations**. The config is only as correct as those inputs.

• **Purview is licensing-/plan-dependent and evolving.** Advanced DLP, auto-labeling, and several retention capabilities require **Microsoft 365 E5 / E5 Compliance**; some features are in preview and cmdlet parameters (notably DLP **simulation/mode** values) change. Capabilities and parameters are **flagged "verify your licensing/tenant,"** never invented.

• **Cmdlet coverage, not portal parity.** Some Purview settings are portal-only or only partially exposed to Security & Compliance PowerShell; the framework manages the attributes the cmdlets expose and flags where a setting must be confirmed in the portal.

• **Records/insider-risk/communication-compliance are out of automated scope.** Their objects encode legal/HR/records determinations and file-plan structure; this framework inventories and reports on the automatable policy types (DLP, labels, retention) and defers those sensitive domains to their own governance.

• **Deploy is high-impact even in test mode.** Creating dozens of objects touches the tenant; that is why deploy is `-WhatIf` first, test/unpublished by default, pilot-scoped, and reversible.

Naming convention (used in every doc and script)

Every artifact this framework creates or produces is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, config, and portal. The standard set:

Artifact	Name
Declarative desired-state config (JSON)	`Automation Config – Purview Ops – Baseline`
Current-state snapshot (Discovery export)	`Automation Snapshot – Tenant State – Current`
DLP/label/retention deploy (safe first state)	`Automation Deploy – Purview Policies – Simulation`
Desired-state / drift validation	`Automation Validate – Desired State – Drift`

Artifact	Name
Policy inventory report	`Automation Report – Policy Inventory – Tenant`
Coverage scorecard	`Automation Report – Coverage Scorecard – Tenant`
Pilot scope	`Automation Pilot – Ops Scope – Stage1`

The standard deployment model

> *Assess the current tenant policy state and the desired-state config → design the config and pilot scope → configure by deploying **into test/simulation/unpublished mode** → **pilot** on a bounded scope and validate against desired state → validate drift is clean and the scorecard is green → expand by enforcing/ publishing under change control. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce).***

2. Requirements Guide — Microsoft Purview Automation Framework

Verify **every** requirement before deploying anything. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant/licensing, Purview roles, the S&C PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, pilot & change control) are in `global-requirements-purview.md` — this guide adds the automation-specific detail.

> ***Licensing/tenant boundary.** The policy types this framework automates are **plan-gated**: advanced DLP (fingerprint/EDM/trainable-classifier conditions), auto-labeling, and several retention capabilities require **Microsoft 365 E5** or **E5 Compliance**; some features and cmdlet parameters (notably the DLP **simulation/mode** values) are **evolving or in preview**. Where this guide flags "verify your licensing/ tenant," confirm the capability and the exact parameter values in your tenant before relying on them.*

2.1 Compliance licensing for the policy types you automate

Description: The framework deploys **DLP**, **sensitivity-label**, and **retention** policy objects. Baseline DLP/labels/retention exist with E3-class licensing; **advanced DLP conditions, auto-labeling, and some retention features require E5 / E5 Compliance** (GR §1). **Why:** A config that references an E5-only condition will fail to deploy on an E3 tenant — the capability isn't there, not a script fault. **Verify:** M365 admin center → `Billing → Licenses`; `OPSAUTO-Health.ps1` reads `subscribedSkus` via Graph and reports whether an E5/E5-Compliance-bearing plan is present, and flags config entries that need it. **Implement:** Assign/purchase the licensing the config's policies require before deploying those policies; scope the config to what your plan supports.

2.2 Purview roles (least-privilege) for each policy type

Description: Deploying and reading each object needs the matching role group: **Compliance Administrator** or **Compliance Data Administrator** for DLP and retention; **Information Protection** (Admin) for sensitivity labels and label policies; **View-Only** equivalents for the read-only

discovery/validate/report tools. Reserve Global Administrator for exceptional cases only. **Why:** Purview uses fine-grained role groups; create/update requires the write role, and the read-only tooling should run under a view-only role (GR §2). **Verify:** Purview portal → `Settings → Roles & scopes / Permissions` — confirm membership. `OPSAUTO-Health.ps1` attempts a benign read of each object type and reports which are permitted. **Implement:** Assign the least-privilege role group per policy type under change control; run read-only tools (Discovery/Validate/Report) with a view-only account, and Deploy with the write role.

2.3 Security & Compliance PowerShell + Graph toolchain

Description: A workstation with the `ExchangeOnlineManagement` module (provides `Connect-IPPSSession` for **all** the `*-DlpCompliancePolicy`, `*-Label`, `*-RetentionCompliancePolicy` cmdlets) and `Microsoft.Graph` (for licensing reads), driven by a modern-auth / MFA admin. **Why:** Every policy object this framework manages is a **Security & Compliance PowerShell** object; Graph is used only for licensing/reporting reads (GR §3). **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, Microsoft.Graph`; `Connect-IPPSSession` and `Connect-MgGraph` succeed. `OPSAUTO-Health.ps1` connects and reports module versions and endpoint reachability. **Implement:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; `Install-Module Microsoft.Graph -Scope CurrentUser`. Keep the module current — the S&C cmdlet surface evolves.

2.4 A reviewed desired-state config (`Automation Config – Purview Ops – Baseline`)

Description: The JSON config that lists the DLP/label/retention policies the framework manages, each with its managed attributes (name, locations/scope, mode, identifying settings). **Its content — which info types, which durations, which labels — comes from compliance/records/legal**, not from the framework. **Why:** The framework is only as correct as the config; the config is the reviewable record of the compliance decision (GR §4, §5). **Verify:** The config validates against the framework's expected shape (`OPSAUTO-Deploy.ps1 -ValidateConfigOnly` parses it and reports structural errors) and every policy's *content* has a named compliance/records/legal owner. **Implement:** Author the config from the approved policy design; capture the approver per policy. Start with a small, high-confidence subset for the pilot.

2.5 In-scope locations/workloads and a classification intent

Description: The **workloads/locations** each policy targets (Exchange, SharePoint, OneDrive, Teams, endpoints) and the **classification/sensitivity** intent driving the labels/DLP/retention (GR §4). **Why:** Every deployed policy keys off its locations and the classification scheme; a mis-scoped policy is a high-impact mistake. **Verify:** The config's `locations`/`scope` per policy match the agreed in-scope workloads; stakeholders have confirmed the classification approach. **Implement:** Define scope and classification intent before authoring config; do not deploy tenant-wide locations during pilot.

2.6 Compliance, records, privacy & legal ownership of policy content

Description: Named owners — from **compliance, records, privacy, security, legal** — for the *content* of each managed policy (retention durations, records intent, DLP sensitive-info-type selection, label

taxonomy) (GR §5). **Why:** These are governance determinations, not technical defaults; the framework configures to them and defers the determination. **Verify:** Each policy in the config maps to a documented decision and a named approver. **Implement:** Capture the determinations and approvals before deploying the corresponding policy.

2.7 A pilot scope, change control, and understanding of the safe-first modes

Description: A bounded **pilot scope** — `Automation Pilot – Ops Scope – Stage1` (a few locations/users/policies) — plus an agreed change window, and a shared understanding that the framework deploys DLP in **test/simulation mode**, labels **unpublished**, and retention **unenforced/pilot-scoped**, and that **enforcement is a separate explicit step** (GR §6). **Why:** DLP, label, and retention changes are high-impact; every solution follows Assess → Design → Configure → **Pilot (simulate)** → Validate → Expand (enforce). **Verify:** A named pilot scope and change approval exist; the team agrees nothing enforces without a deliberate `-Enforce -Commit`. **Confirm the DLP test/simulation mode values your tenant accepts** (2.1) — they are evolving. **Implement:** Define the pilot scope and change window; agree the enforce gate before running any `-Commit`.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Automation Framework

Follow these steps **in order**. All work uses the **Microsoft Purview portal** (`https://purview.microsoft.com`), **Security & Compliance PowerShell** (`Connect-IPPSSession` — every `*-DlpCompliancePolicy`, `*-Label`, `*-RetentionCompliancePolicy` cmdlet), and **Microsoft Graph** (licensing/reporting reads). Every write is gated: scripts run **read-only or** `-WhatIf` **by default** and change state only with `-Commit`; nothing **enforces/publishes** without a separate, explicit `-Enforce -Commit`. Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are reused verbatim.

> ***Model:*** *Assess current state and the config, design the config and pilot, configure by deploying into* ***test/simulation/unpublished mode****,* ***pilot*** *on a bounded scope, validate against desired state, then expand by enforcing under change control. Pattern:* ***Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce).***

Step 1 — Confirm the toolchain, roles, and licensing (Assess)

1. Run `OPSAUTO-Health.ps1` (read-only). It reports: `ExchangeOnlineManagement`/`Microsoft.Graph` module versions, `Connect-IPPSSession`/`Connect-MgGraph` reachability, which policy-object types you can read (a permissions probe per type — Requirement 2.2), and whether an E5/E5-Compliance-bearing plan is

present (Requirement 2.1).

2. Resolve any red item — missing module, failed connect, missing role, missing license — before proceeding.

3. **Pass gate:** health check is green for the policy types your config targets; you can read DLP/label/retention objects and Graph licensing.

Step 2 — Inventory the current tenant state (Assess)

1. Run `OPSAUTO-Discovery.ps1` (read-only). It enumerates existing **DLP policies + their mode**, **sensitivity labels + label policies**, and **retention policies + rules**, and writes the snapshot `Automation Snapshot – Tenant State – Current` (JSON).

2. Review the snapshot: what already exists, in what mode, and where it overlaps the intended config. This is the baseline you reconcile against.

3. **Pass gate:** a current-state snapshot exists and has been reviewed with the policy owners.

Step 3 — Author and validate the desired-state config (Design)

1. Author `Automation Config – Purview Ops – Baseline` (JSON) with the policy owners (Requirement 2.4/2.6): for each DLP/label/retention policy, its **name**, **locations/scope**, target **mode**, and the identifying managed settings. **Content decisions (info types, durations, taxonomy) come from compliance/records/legal** — the config records them.

2. Structurally validate it: `OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -ValidateConfigOnly` (parses the config, checks required fields, flags entries that need E5 — no tenant calls).

3. Define the **pilot scope** `Automation Pilot – Ops Scope – Stage1` — a bounded subset of locations/users and a few policies to prove the flow (Requirement 2.7).

4. **Pass gate:** the config validates structurally, every policy has a named content-owner, and the pilot scope is documented and approved.

Step 4 — Deploy the pilot into safe-first mode (Configure → Pilot)

1. Dry-run the reconcile for the pilot subset:

`OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -WhatIf` It prints the exact `New-/Set-*` cmdlets it *would* run — creates, updates, and mode settings — with **no changes**.

1. Review every planned action. Confirm names match the config, locations are the **pilot** scope (not tenant-wide), and DLP policies are targeted at **test/simulation mode**, labels **unpublished**, retention **unenforced/pilot-scoped**.

2. Apply the safe-first deploy: re-run with `-Commit` (still **no enforcement**). Result: DLP policies exist in **test/simulation mode**, sensitivity labels exist **but are not published**, retention policies exist **scoped to the pilot / unpublished**. Label the effort `Automation Deploy – Purview Policies – Simulation`.

3. **Pass gate:** `OPSAUTO-Discovery.ps1` shows the pilot policies present in the intended safe-first mode. **Rollback:** `OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot`

`-Remove -Commit` removes the objects the framework created (it only touches config-managed objects; it does not delete pre-existing unmanaged policies).

Step 5 — Validate deployed state against desired state (Pilot → Validate)

1. Run `OPSAUTO-Validate.ps1 -ConfigPath .\baseline.json -Scope Pilot` (read-only). It compares each config entry to the deployed object and reports **drift**: missing, extra, **mode-mismatched**, or altered policies, producing `Automation Validate – Desired State – Drift`.

2. For DLP policies in **test/simulation mode**, review the simulation/test results in the Purview DLP portal (what *would* have matched) with the policy owners — this is where a mis-scoped or over-broad rule is caught before it ever enforces. **Verify the simulation view/behavior in your tenant** (2.1).

3. **Pass gate:** validation reports **no unexpected drift** for the pilot, and the DLP test/simulation results are reviewed and accepted by the owners.

Step 6 — Report the pilot posture (Validate)

1. Run `OPSAUTO-Report.ps1 -ConfigPath .\baseline.json -Scope Pilot` (read-only) to produce `Automation Report – Policy Inventory – Tenant` and `Automation Report – Coverage Scorecard – Tenant`: what exists, in what mode, versus what the config expects.

2. Complete the **Testing Guide** gates for the pilot before enforcing anything.

3. **Pass gate:** the scorecard is green for the pilot scope and the inventory matches the config.

Step 7 — Enforce/publish the pilot deliberately (Expand — gated)

1. Only after Steps 5–6 pass, and with change approval, promote the pilot objects to their enforced state:

`OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -Enforce -WhatIf` → review → `-Enforce -Commit`. This moves DLP policies to **Enable** (enforce), **publishes** the label policy, and **publishes/scopes-up** retention — the one and only step that turns on real user/data impact, and it is separate from the safe-first deploy by design.

1. Monitor for user/end-state impact on the pilot scope before widening. **Rollback:** re-run without `-Enforce` (`-Commit`) to return DLP to test mode / unpublish labels / unpublish retention; or `-Remove -Commit` to delete the managed objects.

2. **Pass gate:** the pilot enforces cleanly with no unexpected user impact, validation stays drift-free, and the scorecard remains green.

Step 8 — Expand to full scope (Expand)

1. Widen the config's scope from `Pilot` to the full agreed locations/users, and repeat Steps 4–7 for the full set: deploy safe-first (`-Commit`), validate, report, then enforce (`-Enforce -Commit`) under change control. Never jump straight to enforce at full scope without the safe-first + validate pass.

2. Confirm licensing (2.1) covers every location/user the expanded policies touch.

3. **Pass gate:** full-scope validation is drift-free and the scorecard is green with enforced policies matching the config.

Step 9 — Operationalize the cadence (Expand)

1. Schedule the **read-only chain** `OPSAUTO-Discovery.ps1` → `OPSAUTO-Validate.ps1` → `OPSAUTO-Report.ps1` on a cadence (e.g. daily/weekly) so **drift** — a policy edited in the portal, disabled, un-published, or removed from a label policy — is caught and reported early.

2. Treat every config change as a change-controlled pull-request-style flow: edit config → `-ValidateConfigOnly` → `-WhatIf` → `-Commit` (safe-first) → validate → `-Enforce -Commit`.

3. Hand the scorecard and drift report to compliance/security as the standing evidence and monitoring artifact. **Pass gate:** the cadence is scheduled, and config changes flow only through the gated deploy path.

5. Testing & Validation — Microsoft Purview Automation Framework

Work through every gate. Each has a **method**, an **expected result (pass)**, and a **fail action**. Tests are read-only or use the framework's own `-WhatIf`/safe-first modes — no policy enforces during testing. Do not enforce/publish (Implementation Step 7) until every pilot gate passes; then re-confirm on expand.

> ***Remember:*** *deploying into test/simulation/unpublished mode is *not* enforcement. These gates prove the framework deploys, validates, and reports correctly and that policies land in the intended safe-first state — enforcement is a separate, deliberate step with its own gate.*

Gate 1 — Toolchain, roles, and licensing are ready

• **Method:** `OPSAUTO-Health.ps1`.

• **Pass:** modules present and current; `Connect-IPPSSession`/`Connect-MgGraph` succeed; the policy-object types your config targets are readable; the required E5/E5-Compliance plan is present where the config needs it.

• **Fail:** install/update the module (Requirement 2.3), assign the missing role (2.2) or license (2.1), and retest.

Gate 2 — Current-state snapshot is produced

• **Method:** `OPSAUTO-Discovery.ps1` → `Automation Snapshot – Tenant State – Current`.

• **Pass:** the snapshot enumerates existing DLP policies (with mode), sensitivity labels + label policies, and retention policies + rules.

• **Fail:** resolve the connection/permission error the script reports; confirm the view-only role can read each object type.

Gate 3 — Config validates structurally

• **Method:** `OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -ValidateConfigOnly` (no tenant calls).

• **Pass:** the config parses; required fields present; entries needing E5 are flagged; no structural errors.

• **Fail:** correct the config shape/fields the script names; re-validate before any deploy.

Gate 4 — Dry-run (`-WhatIf`) shows the intended actions only

• **Method:** `OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -WhatIf`.

• **Pass:** the printed `New-/Set-*` actions match the config; scope is the **pilot** (not tenant-wide); DLP targets **test/simulation mode**, labels **unpublished**, retention **unenforced/pilot-scoped**; **no** enforce/publish action appears.

• **Fail:** the plan shows the wrong scope, mode, or an unexpected enforce — fix the config/scope and re-dry-run. Do **not** `-Commit` until the plan is correct.

Gate 5 — Safe-first deploy lands objects in the intended mode

• **Method:** `OPSAUTO-Deploy.ps1 ... -Scope Pilot -Commit`, then `OPSAUTO-Discovery.ps1`.

• **Pass:** the pilot DLP policies exist in **test/simulation mode**, labels exist **unpublished**, retention exists **pilot-scoped/unpublished** — nothing is enforcing.

• **Fail:** re-run discovery to confirm mode; if an object enforced, it means `-Enforce` was passed — return it (Step 7 rollback) and re-deploy safe-first.

Gate 6 — Validation reports no unexpected drift

• **Method:** `OPSAUTO-Validate.ps1 -ConfigPath .\baseline.json -Scope Pilot` → `Automation Validate – Desired State – Drift`.

• **Pass:** every config entry has a matching deployed object in the expected mode; no missing/extra/altered/mode-mismatched items beyond what's intended.

• **Fail:** reconcile — re-deploy the missing/changed object (`-Commit`) or fix the config; investigate any **extra** object (a portal-created policy the config doesn't know about) with its owner.

Gate 7 — DLP test/simulation results are reviewed

• **Method:** in the Purview DLP portal, open the simulation/test results for the deployed DLP policies (what *would* have matched) and review with the policy owners. **Verify the simulation view exists/behaves as expected in your tenant** (Requirement 2.1).

• **Pass:** the simulated matches are as intended — no obviously over-broad or mis-scoped rule — and the owners accept them.

• **Fail:** adjust the rule conditions/scope in the config, re-deploy safe-first, and re-review before enforcing.

Gate 8 — Reports/scorecard reflect reality

• **Method:** `OPSAUTO-Report.ps1 -ConfigPath .\baseline.json -Scope Pilot` → inventory + coverage scorecard.

• **Pass:** the inventory matches the deployed objects and the config; the scorecard is green for the pilot (present, correct mode, in scope).

• **Fail:** address the specific red scorecard item (missing policy, wrong mode, out-of-scope) and re-run.

Gate 9 — Enforcement is deliberate and reversible (only when enforcing)

• **Method:** `OPSAUTO-Deploy.ps1 ... -Scope Pilot -Enforce -WhatIf` → review → `-Enforce -Commit`; then confirm the intended enforce/publish state and confirm rollback works (`-Commit` without `-Enforce` returns to safe-first).

• **Pass:** the enforce step affects only the reviewed pilot objects; rollback cleanly returns DLP to test mode / unpublishes labels+retention.

• **Fail:** if enforce touched more than intended, roll back immediately and re-scope; never expand enforcement until the pilot enforce gate is clean.

Gate 10 — Cadence catches injected drift

• **Method:** as a test, change one deployed policy in the portal (e.g. disable it), then run the read-only chain (`Discovery → Validate → Report`).

• **Pass:** validation flags the change as drift and the scorecard reflects it; restore the policy via `-Commit`.

• **Fail:** confirm the scheduled chain runs under a role that can read the object and that the config entry matches; fix and retest.

Evidence to retain

• `OPSAUTO-Health.ps1` and `OPSAUTO-Discovery.ps1` outputs (before/after).

• Every `-WhatIf` transcript for deploy and enforce.

• The drift report and coverage scorecard for the pilot and the expanded scope.

• The DLP test/simulation review notes and the change approvals (Requirement 2.6/2.7).

*The framework proves policies are **deployed, validated, and evidenced** as configured. Whether the policy *content* (info types, durations, taxonomy) is correct or legally adequate is a **compliance/records/legal determination** — these gates test the automation, not the determination.*

6. Troubleshooting — Microsoft Purview Automation Framework

The failure modes that make automated Purview ops **fail closed, drift, or over-reach** — plus the toolchain, config-validation, deploy, and enforcement issues you'll hit running policy management through Graph + PowerShell. Each: **symptom** → **likely cause** → **fix.** The framework is **safe by default** — every write runs in `-WhatIf` until `-Commit`, and nothing **enforces** without a separate, explicit `-Enforce -Commit` — so most "it did the wrong thing" reports are a config error caught in the dry-run, not a live

incident.

`OPSAUTO-Health.ps1` shows a red item

- **Cause:** a missing module, a failed `Connect-IPPSSession`/`Connect-MgGraph`, a missing role for a policy type, or a missing license.
- **Fix:** resolve the specific red item **before** proceeding — `Install-Module ExchangeOnlineManagement` / `Install-Module Microsoft.Graph` (Requirement 2.3), complete modern-auth, assign the least-privilege role for the policy types you manage (Requirement 2.2), confirm E5/E5-Compliance where the config needs it (Requirement 2.1). Do not run the deploy path until health is green for the types your config targets.

`-ValidateConfigOnly` fails or flags entries

- **Cause:** the desired-state config (`Automation Config – Purview Ops – Baseline`) is malformed, missing a required field (name/type/scope/mode), or contains an entry that needs **E5**.
- **Fix:** this check makes **no tenant calls** — fix the JSON: every policy needs `name`, `type` (`dlp`/`label`/`labelPolicy`/`retention`), `scope` (`Pilot`/`Full`), and its target mode. E5-flagged entries stay out until licensed. Re-run `-ValidateConfigOnly` until clean before any `-WhatIf`.

`-WhatIf` prints changes I didn't expect

- **Cause:** the config's names, locations, or modes don't match intent — e.g. a location is tenant-wide when you meant the pilot subset, or a DLP mode is `Enable` when it should be test/simulation.
- **Fix:** `-WhatIf` is the gate — **review every planned** `New-/Set-*` **line.** Confirm names match the config, locations are the **pilot** scope (not tenant-wide), DLP is targeted at **test/simulation mode**, labels **unpublished**, retention **unenforced/pilot-scoped**. Only then re-run with `-Commit`.

A policy deployed but in the wrong mode / already enforcing

- **Cause:** the safe-first deploy and the enforce step were conflated, or a portal edit changed the mode.
- **Fix:** the design separates them deliberately — `-Commit` (safe-first) creates DLP in **test/simulation mode**, labels** unpublished**, retention** pilot-scoped**; only** `-Enforce -Commit` **turns on real user/data impact.** `OPSAUTO-Validate.ps1` **reports the** MODE-MISMATCH**; move it back to safe-first, then enforce deliberately.** Rollback:** re-run without `-Enforce` (`-Commit`) to return to test mode / unpublish.

`OPSAUTO-Validate.ps1` reports drift

- **Cause:** a policy was edited in the portal, disabled, un-published, removed from a label policy, or a managed-name object exists in the tenant but not in the config (`EXTRA`).

- **Fix:** read the drift categories — **MISSING** (in config, not deployed → re-run deploy),

MODE-MISMATCH (wrong mode → correct via deploy or update the config's expectation), **EXTRA** (investigate with the owner; it may be an out-of-band change). The config is the source of truth; changes flow through the gated deploy path, not the portal. **Verify expected DLP mode values for your tenant.**

`OPSAUTO-Report.ps1` scorecard shows present < expected, or unexpected enforced count

- **Cause A — not deployed yet.** Expected-but-missing policies show as `MISSING` in the inventory.

Fix: run the safe-first deploy for the scope, then re-report.

- **Cause B — enforced count higher than intended.** A DLP policy is in `Enable` mode when the pilot should

still be test/simulation. **Fix:** confirm this was a deliberate `-Enforce -Commit`; if not, return it to test mode. The scorecard's "in expected mode" vs "enforced in tenant" is the safe-first check.

- **Cause C — EXTRA managed-style names.** An `Automation *`-named object exists that the config doesn't

track. **Fix:** reconcile with the owner — bring it into config or remove it via the gated path.

`-Enforce -Commit` caused user/data impact you didn't want

- **Cause:** enforcement was widened before the pilot was reconciled and reviewed.
- **Fix:** enforce is the **one** step that turns on real impact and it is pilot-scoped by design. **Rollback:**

re-run without `-Enforce` (`-Commit`) to return DLP to test mode / unpublish labels / unpublish retention; or `-Remove -Commit` to delete the managed objects (it touches only config-managed objects, never pre-existing unmanaged policies). Reconcile and review the simulation results before re-enforcing.

`-Remove -Commit` didn't delete a policy I expected

- **Cause:** the framework only removes **config-managed** objects; a pre-existing unmanaged policy is left

untouched by design (it never created it).

- **Fix:** this is intentional — the framework will not delete objects it didn't create. Remove genuinely

unmanaged policies deliberately in the portal under change control.

Scheduled read-only chain didn't run / drift caught late

- **Cause:** the cadence (`Discovery → Validate → Report`) wasn't scheduled, or the account's token expired.
- **Fix:** schedule the read-only chain (daily/weekly) so a portal edit, disable, un-publish, or removal is

caught early; confirm the account can complete modern-auth/MFA unattended (or run it interactively on cadence). Drift found late is a monitoring gap, not a tool fault.

`Connect-IPPSSession` / `Connect-MgGraph` fails mid-run

• **Cause:** module not installed, MFA/modern-auth not completed, or a throttled/expired session.

• **Fix:** install the modules, complete the interactive sign-in (Requirement 2.3), and re-run. Scripts are read-only or `-WhatIf` by default, so a failed connect is safe to retry. **No credentials in code.**

When to escalate beyond this solution

• **What a policy should *contain*** — info types, retention durations, label taxonomy, insider-risk conditions → **compliance, records, privacy, and legal**. The framework records and deploys those decisions; it does not make them.

• **Whether to enforce tenant-wide** → the policy owners + change control, after a reconciled, reviewed pilot.

• **A cmdlet or DLP mode value that has genuinely changed** → verify against current Microsoft documentation; DLP modes and policy cmdlets evolve and are licensing-dependent.

7. Best Practices — Automating Microsoft Purview Operations Safely

Operating principles for running Purview policy operations as repeatable, reversible, evidenced code. Grounded in Microsoft's public guidance and the **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand (enforce)** model.

Treat policy as reviewable code

• Keep the **desired-state config** (`Automation Config – Purview Ops – Baseline`) in version control and change it through a review flow: edit → `-ValidateConfigOnly` → `-WhatIf` → `-Commit` (safe-first) → validate → `-Enforce -Commit`. The config, not the portal, is the source of truth.

• One config, many environments — the same config deployed to pilot and production means production is not a hand-rebuilt approximation of what you tested.

Deploy safe-first, enforce deliberately

• **Never let deploy and enforce be the same action.** The framework deploys DLP in **test/simulation mode**, labels **unpublished**, retention **unenforced/pilot-scoped** by default; enforcing is a separate, explicit `-Enforce -Commit`. Keep it that way — it is the single biggest guard against a high-impact mistake.

• Always `-WhatIf` **first** and read the plan. The plan is your last cheap chance to catch a tenant-wide scope, a wrong mode, or an unintended enforce.

• **Review DLP simulation/test results** with the policy owners before enforcing — a rule that looks right in config can match far more (or less) than intended. **Verify the simulation behavior in your tenant**; mode values and the simulation experience evolve.

Pilot small, expand under control

• Prove the whole flow on `Automation Pilot – Ops Scope – Stage1` — a few locations/users/policies — before touching full scope. Pilot-first is not optional for DLP/label/retention changes; they hit real users and data on enforce.

• Expand only after the pilot validates drift-free, the scorecard is green, and the pilot enforce gate is clean. Re-run the safe-first + validate cycle at full scope; don't jump straight to enforce.

Make drift visible

• Schedule the **read-only chain** (Discovery → Validate → Report) so a portal edit, a disabled policy, an unpublished label policy, or a removed rule surfaces as reported drift rather than an audit-day surprise.

• Investigate **extra** objects (policies in the tenant the config doesn't know about) with their owners — either bring them under config management or retire them; unmanaged policies are how drift accumulates.

Manage only what you own — and know the boundary

• The framework touches **only config-managed objects**; it does not delete or alter unmanaged pre-existing policies. Keep that discipline so automation can never surprise you by removing something it didn't create.

• **Records-management file plans, insider-risk, and communication-compliance are out of automated scope** — they encode legal/HR/records determinations. Inventory and report on them if you must, but manage them under their own governance.

Least-privilege and separation

• Run the **read-only tools** (Health, Discovery, Validate, Report) under a **view-only** role; run **Deploy** under the write role for that policy type; reserve Global Administrator for exceptions. The read/write split limits blast radius and makes the scheduled cadence safe.

• Access to policy configuration is sensitive — review role membership periodically and keep the deploy path auditable (every change is a committed config edit + a `-WhatIf`/`-Commit` transcript).

Keep decisions where they belong

• The framework **carries** compliance/records/legal decisions and **proves** they landed; it does not make them. *Which* info types, *which* retention duration, *which* labels — and legal adequacy — stay with **compliance, records, privacy, and legal**. Record the owner per policy in the config.

• Verify the config against **current Microsoft documentation** periodically — Purview cmdlet parameters, DLP mode values, and capability availability change; a config that deployed cleanly last quarter may need updating.

No secrets, ever

• The tools use **interactive modern-auth / MFA** sign-in; **no client secrets or credentials in code or config**. If you later automate unattended runs, use a certificate-based app identity with least-privilege scopes and store the certificate outside the repo — never a secret in the config file.

8. Deliverables Summary — Microsoft Purview Automation Framework

What you receive, what to run, and what "done" looks like.

Documentation

Document	Purpose
1. Solution Overview	What the automation framework is, the config-as-code model, the Purview objects it manages (DLP, labels, retention), what you control, honest limitations, and the naming convention.
2. Requirements Guide	Automation-specific prerequisites (licensing per policy type, least-privilege roles per type, S&C PowerShell + Graph toolchain, a reviewed desired-state config, scope/classification, content ownership, pilot & safe-first modes) — each with Verify / Implement.
3. Implementation Guide	Ordered steps: health-check, discover current state, author + validate config, deploy safe-first, validate drift, report, enforce deliberately, expand, operationalize the cadence — pilot-first with pass/fail gates and rollback.
5. Testing & Validation	Ten gates from "toolchain ready" to "cadence catches injected drift," each with method / pass / fail action.
6. Troubleshooting	The failure modes of config-driven Purview automation — connect/role/licensing errors, mode/parameter drift, unexpected extra objects, enforce mistakes, throttling — and their fixes.
7. Best Practices	Operating principles: policy as code, deploy safe-first + enforce deliberately, pilot small, make drift visible, manage only what you own, least-privilege, keep decisions where they belong, no secrets.

PowerShell tools (5)

All run **read-only or `-WhatIf` by default**; writes require `-Commit`, and **enforcement/publishing requires a separate explicit** `-Enforce -Commit`. No credentials or secrets are embedded — interactive modern-auth / MFA sign-in. All policy objects are **Security & Compliance PowerShell** (`Connect-IPPSSession`); licensing/reporting reads use **Microsoft Graph**.

Script	Role	Default
`OPSAUTO-Health.ps1`	Assess/scope helper: module versions, connectivity, per-policy-type read permission probe, licensing (E5/E5-Compliance) check.	Read-only
`OPSAUTO-Discovery.ps1`	Assess: inventory existing DLP policies (+ mode), sensitivity labels + label policies, retention policies + rules → state snapshot.	Read-only
`OPSAUTO-Deploy.ps1`	Configure/Remediate: reconcile config → tenant. Validate-only, then `-WhatIf`; deploys **safe-first** (DLP test/simulation, labels/retention unpublished) on `-Commit`; enforces only on `-Enforce -Commit`; `-Remove` cleans up managed objects.	`-WhatIf` (writes on `-Commit`; enforce on `-Enforce -Commit`)
`OPSAUTO-Validate.ps1`	Validate: compare deployed objects to the desired-state config → drift report (missing/extra/altered/mode-mismatched), pass/fail.	Read-only
`OPSAUTO-Report.ps1`	Validate: policy inventory + coverage scorecard (present, correct mode, in scope, versus config).	Read-only

Named artifacts created / produced

- Desired-state config: `Automation Config – Purview Ops – Baseline` (JSON, customer-owned).
- Current-state snapshot: `Automation Snapshot – Tenant State – Current`.
- Safe-first deploy effort: `Automation Deploy – Purview Policies – Simulation`.
- Drift validation: `Automation Validate – Desired State – Drift`.
- Reports: `Automation Report – Policy Inventory – Tenant`, `Automation Report – Coverage Scorecard – Tenant`.
- Pilot scope: `Automation Pilot – Ops Scope – Stage1`.

Definition of done

- Health check **green** for the policy types the config targets (modules, connectivity, roles, licensing).
- A reviewed **desired-state config** that validates structurally, with a named content-owner per policy.
- The config **deployed safe-first** to the pilot — DLP in **test/simulation mode**, labels **unpublished**, retention **pilot-scoped** — and confirmed by discovery.

• **Validation drift-free** and the **coverage scorecard green** for the pilot; DLP test/simulation results reviewed and accepted.

• Enforcement performed **deliberately** (`-Enforce -Commit`) only after the pilot passed, then **expanded** to full scope through the same safe-first → validate → enforce cycle.

• The **read-only cadence** (Discovery → Validate → Report) scheduled so drift is caught, and config changes flowing only through the gated deploy path.

Boundary: *this framework delivers repeatable deployment, validation, and evidence for Purview policy objects.* ***The policy *content* — which sensitive info types, which retention durations, which labels, and whether a policy is legally adequate — is deferred to your compliance, records, privacy, and legal functions.*** *Records-management, insider-risk, and communication-compliance objects are* ***out of automated scope.*** *Feature availability and cmdlet parameters (notably DLP simulation/mode values) are* ***licensing-/ tenant-dependent and evolving — verify against your entitlements and current Microsoft documentation.***

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSAUTO-Deploy.ps1

Purpose: Reads a declarative desired-state config (Automation Config - Purview Ops - Baseline) and reconciles it into the tenant: creates/updates the DLP, sensitivity-label, and retention policies it lists. Runs -WhatIf by default (prints the exact New-/Set-* it would run, no changes). On -Commit it deploys SAFE-FIRST: DLP into test/simulation mode, labels/retention UNPUBLISHED. It ENFORCES (DLP Enable, publish labels/retention) ONLY on the separate, explicit -Enforce -Commit. -Remove deletes only config-managed objects. HOW TO RUN 1. structural check (no tenant calls): .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -ValidateConfigOnly 2. dry run: .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -WhatIf 3. safe-first deploy: .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -Commit 4. enforce (deliberate, gated): .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -Enforce -Commit rollback: .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -Remove -Commit Requires ExchangeOnlineManagement (Connect-IPPSSession) + the write role per policy type. No secrets. EXPECTED RESULTS -WhatIf prints planned actions and makes NO changes. -Commit reports created/updated objects in SAFE-FIRST mode. -Enforce -Commit reports objects moved to enforce/publish. VERIFY the DLP test/simulation mode value your tenant accepts before -Commit. .NOTES Purview category — Automation Framework. Safe-by-default (-WhatIf). Enforcement is a separate -Enforce gate. DLP mode values / cmdlet parameters are evolving/licensing-dependent — verify against your tenant and current Microsoft documentation. No hardcoded credentials or secrets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ConfigPath, [ValidateSet('Pilot','Full')][string]$Scope = 'Pilot', [switch]$ValidateConfigOnly, # parse + structurally validate the config; no tenant calls [switch]$WhatIf, # default posture when neither -Commit nor -Remove is set [switch]$Commit, # apply changes SAFE-FIRST (no enforcement unless -Enforce also set) [switch]$Enforce, # with -Commit: move objects to enforce/publish (HIGH-IMPACT, deliberate) [switch]$Remove, # remove config-managed objects (requires -Commit to actually delete) [string]$UserPrincipalName, The DLP "test/simulation" mode value varies by tenant/version — override here after verifying yours. [string]$DlpTestMode = 'TestWithoutNotifications')

How to run: 1. structural check (no tenant calls): .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -ValidateConfigOnly 2. dry run: .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -WhatIf 3. safe-first deploy: .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -Commit 4. enforce (deliberate, gated): .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -Enforce -Commit rollback: .\OPSAUTO-Deploy.ps1 -ConfigPath .\baseline.json -Scope Pilot -Remove -Commit Requires ExchangeOnlineManagement (Connect-IPPSSession) + the write role per policy type. No secrets. EXPECTED RESULTS -WhatIf prints planned actions and makes NO changes. -Commit reports created/updated objects in SAFE-FIRST mode. -Enforce -Commit reports objects moved to enforce/publish. VERIFY the DLP test/simulation mode value your tenant accepts before -Commit. .NOTES Purview category — Automation Framework. Safe-by-default (-WhatIf). Enforcement is a separate -Enforce gate. DLP mode values / cmdlet parameters are evolving/licensing-dependent — verify against your tenant and current Microsoft documentation. No hardcoded credentials or secrets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ConfigPath, [ValidateSet('Pilot','Full')][string]$Scope = 'Pilot', [switch]$ValidateConfigOnly, # parse + structurally validate the config; no tenant calls [switch]$WhatIf, # default posture when neither -Commit nor -Remove is set [switch]$Commit, # apply changes SAFE-FIRST (no enforcement unless -Enforce also set)

[switch]$Enforce, # with -Commit: move objects to enforce/publish (HIGH-IMPACT, deliberate) [switch]$Remove, # remove config-managed objects (requires -Commit to actually delete) [string]$UserPrincipalName, The DLP "test/simulation" mode value varies by tenant/version — override here after verifying yours. [string]$DlpTestMode = 'TestWithoutNotifications')

OPSAUTO-Discovery.ps1

Purpose: Enumerates the Purview policy objects the framework manages — DLP compliance policies (and their MODE), sensitivity labels, label policies, and retention (data lifecycle) policies + rules — with no changes, and writes a JSON snapshot (Automation Snapshot - Tenant State - Current) that Validate and Report consume. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Discovery.ps1 -OutFile .\snapshot.json Requires ExchangeOnlineManagement (Connect-IPPSSession). Interactive modern-auth / MFA sign-in; no secrets in code. EXPECTED RESULTS Console counts per object type and a JSON snapshot at -OutFile. Each DLP policy shows its Mode (e.g. Enable vs a test/simulation value — VERIFY the exact values in your tenant). Non-fatal read errors per type are reported and skipped. .NOTES Purview category — Automation Framework. Read-only. DLP mode values and cmdlet parameters are evolving/ licensing-dependent; confirm against your entitlements and current Microsoft documentation. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$OutFile = ".\Automation-Snapshot-Tenant-State-Current.json")

How to run: powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Discovery.ps1 -OutFile .\snapshot.json Requires ExchangeOnlineManagement (Connect-IPPSSession). Interactive modern-auth / MFA sign-in; no secrets in code. EXPECTED RESULTS Console counts per object type and a JSON snapshot at -OutFile. Each DLP policy shows its Mode (e.g. Enable vs a test/simulation value — VERIFY the exact values in your tenant). Non-fatal read errors per type are reported and skipped. .NOTES Purview category — Automation Framework. Read-only. DLP mode values and cmdlet parameters are evolving/ licensing-dependent; confirm against your entitlements and current Microsoft documentation. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$OutFile = ".\Automation-Snapshot-Tenant-State-Current.json")

OPSAUTO-Health.ps1

Purpose: Reports whether the workstation and account are ready to run the Purview automation framework, with no changes: ExchangeOnlineManagement / Microsoft.Graph module presence + versions, Security & Compliance PowerShell (Connect-IPPSSession) and Graph reachability, a per-policy-type READ permission probe (DLP / sensitivity labels / retention), and whether an E5 / E5-Compliance-bearing plan is present. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Health.ps1 Requires ExchangeOnlineManagement (Connect-IPPSSession) and, for the licensing section, Microsoft.Graph with delegated Organization.Read.All / Directory.Read.All. Interactive modern-auth / MFA sign-in; no secrets in code. EXPECTED RESULTS A readiness summary: modules present Y/N + version, IPPS + Graph reachable Y/N, each policy type readable Y/N, and an E5/E5-Compliance plan present Y/N. Any licensing/tenant-dependent item is flagged "verify your licensing/tenant", not asserted. Green across the board = ready to run Discovery/Deploy. .NOTES Purview category — Automation Framework. Read-only. Feature/parameter availability is licensing-/tenant-dependent and evolving; confirm against your entitlements and current Microsoft documentation. > [CmdletBinding()] param([string]$UserPrincipalName, # optional admin UPN passed to Connect-* prompts [switch]$SkipGraphLicensing # skip the Graph licensing section if Graph is unavailable)

How to run: powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Health.ps1 Requires ExchangeOnlineManagement (Connect-IPPSSession) and, for the licensing section, Microsoft.Graph with delegated Organization.Read.All / Directory.Read.All. Interactive modern-auth / MFA sign-in; no secrets in code. EXPECTED RESULTS A readiness summary: modules present Y/N + version, IPPS + Graph reachable Y/N, each policy type readable Y/N, and an E5/E5-Compliance plan present Y/N. Any licensing/tenant-dependent item is flagged "verify your licensing/tenant", not asserted. Green across the board = ready to run Discovery/Deploy. .NOTES Purview category — Automation Framework. Read-only. Feature/parameter availability is licensing-/tenant- dependent and evolving; confirm against your entitlements and current Microsoft documentation. > [CmdletBinding()] param([string]$UserPrincipalName, # optional admin UPN passed to Connect-* prompts [switch]$SkipGraphLicensing # skip the Graph licensing section if Graph is unavailable)

OPSAUTO-Report.ps1

Purpose: Produces two standing evidence artifacts from the live tenant (or a captured snapshot) against the desired-state config (Automation Config - Purview Ops - Baseline): "Automation Report - Policy Inventory - Tenant" (every managed DLP/label/retention object, its mode/enable state, and whether the config expects it) and "Automation Report - Coverage Scorecard - Tenant" (counts by type, % of expected policies present, % in the expected safe-first mode, and enforced count). Read-only - changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Report.ps1 -ConfigPath .\baseline.json -Scope Pilot Optional: -SnapshotPath .\snapshot.json to report against a Discovery snapshot instead of live; -OutDir to set where the CSV/JSON land. Requires ExchangeOnlineManagement (Connect-IPPSSession) for live mode. No secrets. EXPECTED RESULTS A printed inventory table + scorecard, plus a CSV inventory and a JSON scorecard on disk. PASS signal: the scorecard's "present vs expected" is 100% for the scope and every present policy is in the mode the config expects (safe-first until deliberately enforced). VERIFY expected DLP mode values for your tenant. .NOTES Purview category - Automation Framework. Read-only. DLP mode values evolve/are licensing-dependent; confirm expected values against your tenant and current Microsoft documentation. No client names; no secrets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ConfigPath, [ValidateSet('Pilot','Full')][string]$Scope = 'Pilot', [string]$SnapshotPath, [string]$UserPrincipalName, [string]$OutDir = ".")

How to run: powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Report.ps1 -ConfigPath .\baseline.json -Scope Pilot Optional: -SnapshotPath .\snapshot.json to report against a Discovery snapshot instead of live; -OutDir to set where the CSV/JSON land. Requires ExchangeOnlineManagement (Connect-IPPSSession) for live mode. No secrets. EXPECTED RESULTS A printed inventory table + scorecard, plus a CSV inventory and a JSON scorecard on disk. PASS signal: the scorecard's "present vs expected" is 100% for the scope and every present policy is in the mode the config expects (safe-first until deliberately enforced). VERIFY expected DLP mode values for your tenant. .NOTES Purview category - Automation Framework. Read-only. DLP mode values evolve/are licensing-dependent; confirm expected values against your tenant and current Microsoft documentation. No client names; no secrets. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ConfigPath, [ValidateSet('Pilot','Full')][string]$Scope = 'Pilot', [string]$SnapshotPath, [string]$UserPrincipalName, [string]$OutDir = ".")

OPSAUTO-Validate.ps1

Purpose: Reconciles the desired-state config (Automation Config - Purview Ops - Baseline) against what is actually deployed and reports DRIFT: MISSING (in config, not in tenant), MODE-MISMATCH (present

but wrong mode/enable state), and EXTRA (a managed-name object in the tenant not tracked... plus flags config-named objects that changed). Produces Automation Validate - Desired State - Drift. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Validate.ps1 -ConfigPath .\baseline.json -Scope Pilot Optionally -SnapshotPath .\snapshot.json to validate against a captured snapshot instead of live. Requires ExchangeOnlineManagement (Connect-IPPSSession) for live mode. No secrets. EXPECTED RESULTS A per-policy PASS/DRIFT line and an overall result. Exit-friendly: writes a JSON drift report and returns non-zero drift count in $LASTEXITCODE-style summary. VERIFY the expected DLP mode value for your tenant (the config's 'expectMode'). .NOTES Purview category — Automation Framework. Read-only. DLP mode values evolve/are licensing-dependent; confirm expected values against your tenant and current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ConfigPath, [ValidateSet('Pilot','Full')][string]$Scope = 'Pilot', [string]$SnapshotPath, # optional: validate against a Discovery snapshot instead of connecting live [string]$UserPrincipalName, [string]$OutFile = ".\Automation-Validate-Desired-State-Drift.json")

How to run: powershell -ExecutionPolicy Bypass -File .\OPSAUTO-Validate.ps1 -ConfigPath .\baseline.json -Scope Pilot Optionally -SnapshotPath .\snapshot.json to validate against a captured snapshot instead of live. Requires ExchangeOnlineManagement (Connect-IPPSSession) for live mode. No secrets. EXPECTED RESULTS A per-policy PASS/DRIFT line and an overall result. Exit-friendly: writes a JSON drift report and returns non-zero drift count in $LASTEXITCODE-style summary. VERIFY the expected DLP mode value for your tenant (the config's 'expectMode'). .NOTES Purview category — Automation Framework. Read-only. DLP mode values evolve/are licensing-dependent; confirm expected values against your tenant and current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ConfigPath, [ValidateSet('Pilot','Full')][string]$Scope = 'Pilot', [string]$SnapshotPath, # optional: validate against a Discovery snapshot instead of connecting live [string]$UserPrincipalName, [string]$OutFile = ".\Automation-Validate-Desired-State-Drift.json")